

Face Recognition System - Interview Defense Cheat Sheet

High-Yield Technical Explanations, Mathematical Formulations, Architecture Trade-Offs & Scaling

Q1: Why do we use Cosine Similarity instead of Euclidean Distance?

- **Direction vs. Magnitude:** Face embeddings are trained via angular margin loss (ArcFace) mapping identities to directional positions on a 512D hypersphere. Angle reflects identity, whereas vector magnitude reflects image brightness, lighting intensity, or sensor artifacts.
- **L2-Norm Efficiency:** All embeddings are L2-normalized to unit length ($\|e\| = 1.0$). Cosine similarity simplifies directly into the vector dot product:

$$\text{CosineSim}(q, e) = (q \cdot e) / (\|q\| * \|e\|) \implies q \cdot e$$

Avoids computing square roots and divisions during 1:N real-time matrix matching ($q \cdot E^T$).

- **Mathematical Relation:** Euclidean distance squared on unit vectors is proportional: $d^2 = 2 - 2(q \cdot e)$. Cosine similarity provides an intuitive, bounded confidence scale $[-1.0, 1.0]$.

Q2: What exactly is a Face Embedding, and how does ArcFace work?

- **512D Semantic Vector:** A compact continuous 512-dimensional vector extracted by ResNet-50 where facial geometry is mapped so identical people have small angular distance and different people have large angular distance.
- **ArcFace Innovation:** Traditional Softmax separates classes using linear hyperplanes without enforcing tight intra-class clustering. ArcFace injects an Additive Angular Margin $m = 0.5$ directly to target ground-truth angle θ_y :

$$\cos(\theta_y + m) \text{ with scale } s = 64$$

Because cosine is monotonically decreasing on $[0, \pi]$, adding margin m penalizes angle, forcing embeddings into tight geodesic cones.

Q3: How did you select the operating threshold ($\tau = 0.60 / 0.34$), and what is the FAR/FRR trade-off?

- **Zero-Leakage Tuning:** Tuned strictly on validation split (80 known queries, 80 unknown queries) across 81 threshold increments (0.10 to 0.90).
- **Optimal Metrics:** At $\tau = 0.34$: FAR = 0.0%, Top-1 Accuracy = 100.0%. Equal Error Rate (EER) where FAR == FRR occurred at $\tau = 0.31$.
- **Production Choice ($\tau = 0.60$):** Higher threshold (0.60) prioritizes security in biometric access control. Intruders are strictly blocked (FAR $\rightarrow$ 0.0), even if genuine users occasionally face false rejections under bad lighting (FRR).

$$\text{FAR} = \text{False Accepts} / \text{Total Impostor Attempts} \quad | \quad \text{FRR} = \text{False Rejects} / \text{Total Genuine Attempts}$$

High Security ($\tau \geq 0.60$): Low FAR, higher FRR $\iff$ High Convenience ($\tau \leq 0.20$): Low FRR, higher FAR

Q4: Why did your test set achieve 100% accuracy, and is that realistic for production?

- **Engineering Honesty:** 100% accuracy was achieved due to: (1) Small gallery size ($N = 40$), and (2) ArcFace's huge angular margin (genuine pairs: 0.55 - 0.85; impostor pairs: < 0.18). Threshold 0.34/0.60 easily separates them.
- **Degradation at Scale:** In open-set matching with $N = 100,000$ enrolled people, extreme value theory dictates that the maximum impostor similarity among random dot products increases significantly: $\max(q \cdot e_i)$.
- **Production Scaling:** At scale, multi-factor verification, dynamic score calibration, and strict thresholds ($\tau \geq 0.60$) are required.

Q5: How are edge cases (no face, multiple faces, corrupt inputs) handled?

- **No Face Detected:** If RetinaFace confidence < 0.50 or box $< 30\text{px}$, system gracefully returns empty list without throwing exceptions. Matcher marks candidate as 'no_face' and safely rejects.
- **Multiple Faces:** Enrollment: Selects largest bounding box by pixel area ($w * h$) as the intended subject. Identification: Detects, aligns, and matches all faces simultaneously in a vectorized loop.
- **Corrupt Images:** Handled via try-except in cv2 decoding; raises clear 400 Bad Request error instead of silent server death.

Q6: How would you scale this system from 40 to 100,000 enrolled identities?

- **FAISS Sub-Linear Search:** Brute-force scan takes $O(N * d)$. For 100,000 vectors, linear search creates latency bottlenecks. We replace it with FAISS HNSW (Hierarchical Navigable Small World) or IVF-PQ graphs, dropping retrieval to $O(\log N)$ ($< 5\text{ms}$).
- **Vector Databases:** Decouple storage into dedicated vector engines (Milvus, Qdrant, or Pinecone) with gRPC microservice separation.
- **Score Calibration:** Apply T-norm / Z-norm score normalization to adjust threshold dynamically per individual, preventing dense cluster bias.

Q7: What are the primary production failure modes and mitigations?

- **Spoofing / Presentation Attacks:** Printed photos, phone screens, 3D masks fool 2D RGB. Mitigation: Passive liveness detection (texture analysis, reflection detection) or IR/Depth sensors.
- **Severe Pose / Yaw (>45 deg):** Affine landmark alignment fails when eye landmarks are occluded. Mitigation: Multi-pose enrollment (frontal, +30 deg, -30 deg) and 3D face mesh normalization.
- **Motion Blur / Poor Light:** Loss of high-frequency spatial features. Mitigation: Pre-capture Laplacian variance blur detector (rejecting frames with var < 100) before neural inference.

Code Nimbus Solutions AI/ML Intern - Core JD Competency Checklist

[x] FastAPI Microservices:	Async endpoints, Pydantic schemas, threadpool execution (asyncio.to_thread), OpenAPI /docs.
[x] Vector Embeddings & RAG:	512D unit-norm vectors, Cosine similarity = dot product, FAISS HNSW sub-linear indexing.
[x] Multimodal Vision Models:	RetinaFace (localization + 5-point landmarks) + ArcFace (512D deep facial semantics).
[x] Generative AI & LLMs:	Gemini 2.5 Flash SDK (google-genai) for automated visual audit reporting and analysis.
[x] Production Observability:	Prometheus latency histograms, OpenTelemetry distributed tracing, Grafana dashboards.